DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Red neuronal convolucional ResNet-100 + API REST

Clasificación de imágenes con una red residual de 100 capas en PyTorch, expuesta mediante una API REST con Django REST Framework y consumida por un cliente de línea de comandos.

Funcionamiento, arquitectura y procedimientos de operación · Octubre 2026

AUTOR

Marlon E. Figueroa



100	4,69 M	6	72,06 %
CAPAS CON PESOS	PARÁMETROS	ENDPOINTS REST	PRECISIÓN ACTUAL (CIFAR-10)

Contenido

1 **Introducción**
- 1.1 Propósito
- 1.2 Componentes
- 1.3 Tecnologías

2 **Estructura y arquitectura general**
- 2.1 Cómo se relacionan los componentes
- 2.2 Organización de carpetas

3 **La red neuronal ResNet-100**
- 3.1 Conceptos básicos y su formulación
- 3.2 Bloque residual
- 3.3 Arquitectura completa y parámetros
- 3.4 Conteo de las 100 capas
- 3.5 Atajos sin parámetros
- 3.6 Inicialización de pesos

4 **Datos y preprocesamiento**

5 **Entrenamiento**

6 **Inferencia (predicción)**

7 **API REST con Django REST Framework**

8 **Cliente de la API**

9 **Procedimientos paso a paso**

10 **Pruebas automatizadas**

11 **Estado actual del proyecto**

12 **Solución de problemas**

13 **Limitaciones y posibles mejoras**

14 **Glosario**

1. Introducción

1.1 Propósito

Este proyecto implementa un sistema completo de **clasificación de imágenes**: dada una fotografía, determina a qué categoría pertenece (por ejemplo, *avión*, *gato* o *barco*) y con qué probabilidad. Para ello combina tres piezas: una red neuronal convolucional profunda que aprende a reconocer patrones visuales, un servicio web que pone esa red a disposición de otras aplicaciones, y un cliente que facilita el uso del servicio.

La red elegida es una **ResNet de 100 capas** (ResNet-100). Las redes residuales resuelven el problema principal de las redes muy profundas —que dejan de aprender cuando se apilan demasiadas capas— mediante *conexiones de atajo* que permiten que la señal y el gradiente fluyan sin degradarse a través de toda la profundidad.

1.2 Componentes

Carpeta	Componente	Responsabilidad
`cnn/`	Red neuronal (PyTorch)	Define la ResNet-100, carga los datos, entrena el modelo, guarda el *checkpoint* y realiza predicciones.
`api/`	API REST (Django + DRF)	Recibe imágenes por HTTP, las clasifica con la red, guarda el resultado en base de datos y lo devuelve en JSON.
`cliente/`	Cliente de línea de comandos	Envía imágenes a la API y muestra las respuestas. También se puede usar como librería (`ClienteCNN`).
`tests/`	Pruebas de la red	Verifica el número de capas, la forma de las salidas y el funcionamiento del predictor.
`checkpoints/`	Modelos entrenados	Contiene `resnet100.pt` , el archivo con los pesos aprendidos.
`data/`	Conjuntos de datos	Almacena CIFAR-10 descargado automáticamente (excluido de Git).

1.3 Tecnologías

Tecnología	Versión mínima	Uso en el proyecto
Python	3.10+ (probado en 3.14)	Lenguaje de todo el proyecto.
PyTorch (`torch`)	2.2	Definición y entrenamiento de la red, inferencia.
torchvision	0.17	Dataset CIFAR-10, `ImageFolder` y transformaciones de imagen.
Pillow	10.0	Lectura y conversión de imágenes a RGB.
NumPy	1.26	Dependencia numérica de PyTorch/torchvision.
Django	5.0	Framework web, ORM, administración y almacenamiento de archivos.
Django REST Framework	3.15	Serializadores, vistas y enrutador de la API REST.
requests	2.31	Peticiones HTTP desde el cliente.
pytest	8.0	Ejecución de las pruebas de la red.
SQLite	(incluido en Python)	Base de datos de la API (`api/db.sqlite3`).

2. Estructura y arquitectura general

2.1 Cómo se relacionan los componentes

El sistema tiene dos fases. En la fase de **entrenamiento**, el script `cnn/train.py` lee imágenes etiquetadas, ajusta los pesos de la red y guarda el mejor resultado en `checkpoints/resnet100.pt`. En la fase de **servicio**, la API carga ese checkpoint una sola vez y lo usa para clasificar cada imagen que recibe.

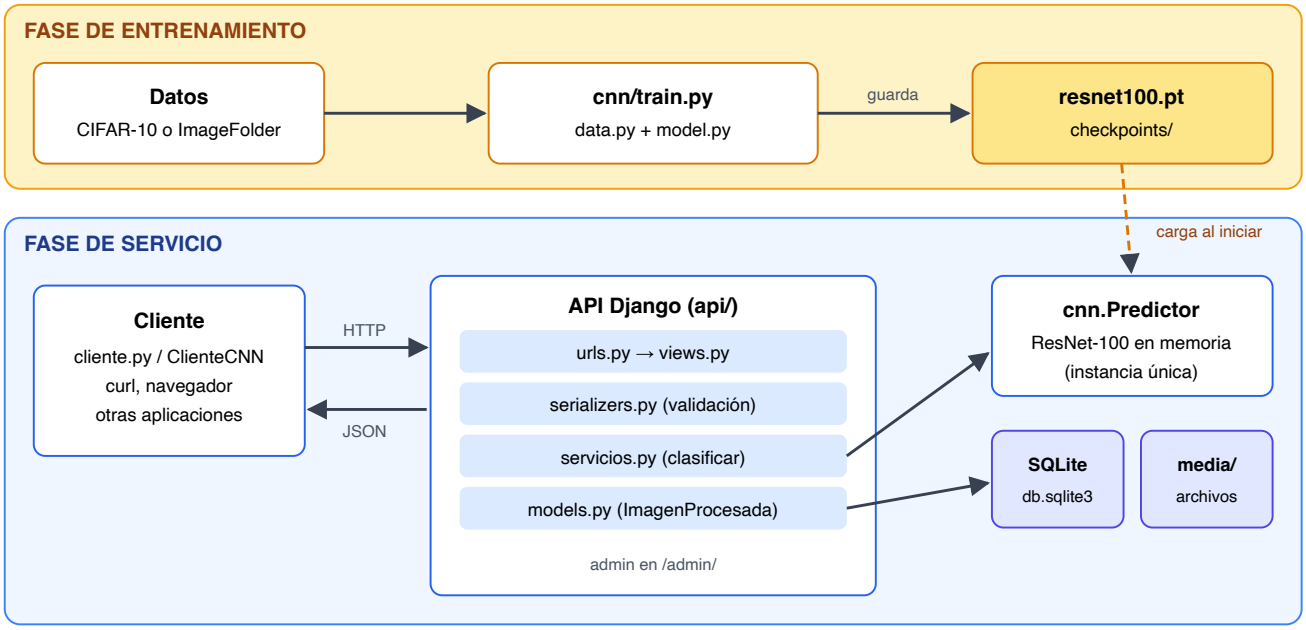


Figura 1. Relación entre la fase de entrenamiento y la fase de servicio.

2.2 Organización de carpetas

```
red_neuronal_convulocional/
├── cnn/                        # Paquete de la red neuronal
│   ├── config.py               # Rutas, clases de CIFAR-10, media y desviación
│   ├── model.py                # ResNet100, BloqueResidual, AtajoSinParametros
│   ├── data.py                 # Transformaciones y DataLoaders
│   ├── train.py                # Script de entrenamiento (python -m cnn.train)
│   ├── predictor.py            # Clase Predictor y obtener_predictor()
│   ├── predict.py              # Predicción por línea de comandos
│   └── utils.py                # Selección de dispositivo (CUDA / MPS / CPU)
├── api/                        # Proyecto Django
│   ├── manage.py
│   ├── config/                 # settings.py, urls.py, wsgi.py, asgi.py
│   ├── procesamiento/          # App: models, serializers, views, urls, servicios, admin, tests
│   ├── db.sqlite3              # Base de datos (se crea con migrate)
│   └── media/imagenes/         # Imágenes subidas, organizadas por AAAA/MM/DD
├── cliente/
│   └── cliente.py              # CLI y clase ClienteCNN
├── tests/
│   └── test_model.py           # Pruebas de la red (pytest)
├── checkpoints/
│   └── resnet100.pt            # Modelo entrenado
├── data/                       # CIFAR-10 descargado
├── docs/                       # Esta documentación
├── requirements.txt
├── pytest.ini
└── README.md
```

El archivo `.gitignore` excluye del repositorio el entorno virtual, los datos descargados (`data/`), los pesos entrenados (`checkpoints/*.pt`), las imágenes subidas (`api/media/`) y la base de datos (`api/db.sqlite3`). Por eso, al clonar el proyecto en otra máquina es necesario volver a entrenar (o copiar el checkpoint) y ejecutar las migraciones.

3. La red neuronal ResNet-100

La red está definida en `cnn/model.py`. Recibe una imagen en color (3 canales) y devuelve un vector de *logits* $\mathbf{z} \in \mathbb{R}^K$, un puntaje por cada una de las K clases. Aplicando la función softmax a ese vector se obtienen las probabilidades de cada clase (sección 6).

En las fórmulas de este capítulo se usa la siguiente notación: $\mathbf{x}$ es un tensor de activaciones con forma $C \times H \times W$ (canales × alto × ancho), C_{in} y C_{out} son los canales de entrada y de salida de una capa, k es el tamaño del filtro y s el paso.

3.1 Conceptos básicos y su formulación

Elemento	Qué hace
Convolución 3×3 (`nn.Conv2d`)	Desliza pequeños filtros de 3×3 píxeles sobre la imagen para detectar patrones locales: bordes y colores en las primeras capas, texturas y formas en las intermedias, partes de objetos en las últimas.
BatchNorm (`nn.BatchNorm2d`)	Normaliza las activaciones de cada canal para que tengan media cercana a 0 y varianza cercana a 1. Acelera y estabiliza el entrenamiento.
ReLU	Función de activación $\mathrm{ReLU}(x) = \max(0,\, x)$. Introduce no linealidad para que la red pueda aprender relaciones complejas.
Paso (stride) 2	La convolución salta un píxel de cada dos, lo que reduce a la mitad el alto y el ancho del mapa de características.
Conexión residual	Suma la entrada de un bloque a su salida. El bloque solo necesita aprender la *diferencia* (el residuo) respecto a la identidad, lo que permite entrenar redes muy profundas.
Average pooling adaptativo	Promedia cada mapa de características en un único valor, sin importar su tamaño. Gracias a esto la red acepta imágenes de cualquier resolución.
Capa lineal (`nn.Linear`)	Combina las 128 características finales para producir un puntaje por clase.

CONVOLUCIÓN 3×3

$$Y_{o,\,i,\,j} \;=\; \sum_{c=1}^{C_{in}} \sum_{u=-1}^{1} \sum_{v=-1}^{1} W_{o,\,c,\,u,\,v}\; X_{c,\; s\,i+u,\; s\,j+v}$$

donde:

- $Y_{o,i,j}$ es el valor del canal de salida o en la posición (i, j);
- $W_{o,c,u,v}$ son los pesos del filtro (9 por cada par de canales de entrada y salida);
- s es el paso: 1 dentro de una etapa y 2 en el primer bloque de las etapas 2, 3 y 4.

Las convoluciones no tienen sesgo (`bias=False`) porque la BatchNorm siguiente ya aporta un desplazamiento β.

TAMAÑO ESPACIAL DE LA SALIDA

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1$$

Con $k = 3$ y relleno $p = 1$: si $s = 1$, entonces $H_{out} = H_{in}$ (se conserva el tamaño); si $s = 2$, entonces $H_{out} = H_{in}/2$ para H_{in} par. Por ejemplo, $\lfloor (32 + 2 - 3)/2 \rfloor + 1 = 16$. Lo mismo aplica al ancho W.

NORMALIZACIÓN POR LOTES (BATCHNORM)

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \qquad y = \gamma\,\hat{x} + \beta$$

donde:

- $\mu_{\mathcal{B}}$ y $\sigma_{\mathcal{B}}^2$ son la media y la varianza del canal calculadas sobre el lote durante el entrenamiento; en evaluación se usan sus promedios móviles;
- γ (escala) y β (desplazamiento) son parámetros aprendidos, uno de cada uno por canal;
- $\varepsilon = 10^{-5}$ evita divisiones entre cero.

AVERAGE POOLING ADAPTATIVO Y CAPA LINEAL

$$h_c = \frac{1}{H\,W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_{c,i,j}, \qquad \mathbf{z} = W_{\text{fc}}\,\mathbf{h} + \mathbf{b}, \quad W_{\text{fc}} \in \mathbb{R}^{K \times 128},\ \mathbf{b} \in \mathbb{R}^{K}$$

El pooling convierte el tensor final de $128 \times 4 \times 4$ en un vector $\mathbf{h}$ de 128 valores, sin importar H y W. La capa lineal transforma ese vector en los K logits $\mathbf{z}$.

3.2 Bloque residual

La unidad que se repite en la red es `BloqueResidual`. Contiene dos convoluciones 3×3, cada una seguida de BatchNorm. La entrada original se suma al resultado justo antes de la última ReLU:

SALIDA DE UN BLOQUE RESIDUAL

$$\mathbf{y} = \text{ReLU}\big(\mathcal{F}(\mathbf{x}) + \mathcal{S}(\mathbf{x})\big), \qquad \mathcal{F}(\mathbf{x}) = \text{BN}_2\Big(W_2 * \text{ReLU}\big(\text{BN}_1(W_1 * \mathbf{x})\big)\Big)$$

donde:

- $\mathcal{F}$ es la *rama residual*: las dos convoluciones W_1, W_2 con sus BatchNorm ($*$ denota convolución);
- $\mathcal{S}$ es el *atajo*: la identidad $\mathcal{S}(\mathbf{x}) = \mathbf{x}$ dentro de una etapa, o el atajo sin parámetros de la sección 3.5 al cambiar de etapa.

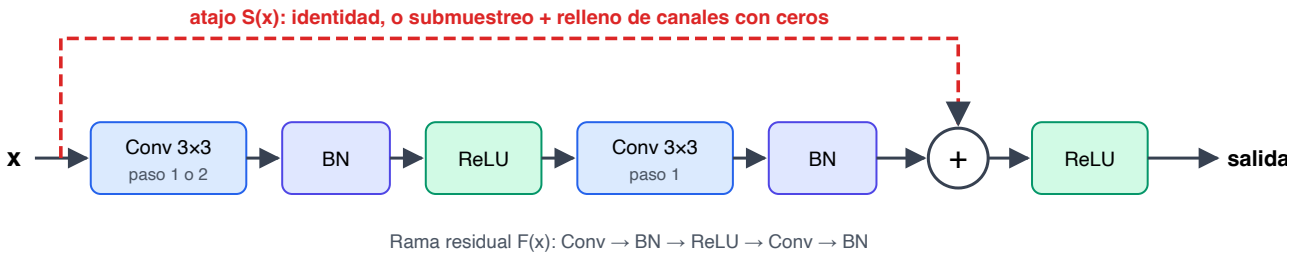


Figura 2. Estructura de un bloque residual (`BloqueResidual`).

POR QUÉ FUNCIONA: EL GRADIENTE A TRAVÉS DEL ATAJO

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \left(I + \frac{\partial \mathcal{F}}{\partial \mathbf{x}} \right)$$

Expresión simplificada (atajo identidad, sin considerar la ReLU final). El término I garantiza que el gradiente de la pérdida $\mathcal{L}$ llegue a las primeras capas aunque $\partial \mathcal{F} / \partial \mathbf{x}$ sea pequeño. En una red sin atajos el gradiente es un producto de 100 factores que tiende a desvanecerse.

3.3 Arquitectura completa y parámetros

La red apila una convolución inicial, cuatro **etapas** de bloques residuales y un clasificador final. Cada etapa duplica el número de canales y (salvo la primera) reduce a la mitad la resolución espacial. Con un ancho base a (`ancho_base`, 16 por defecto), la etapa $e \in \{1, 2, 3, 4\}$ tiene $C_e = a \cdot 2^{e-1}$ canales.

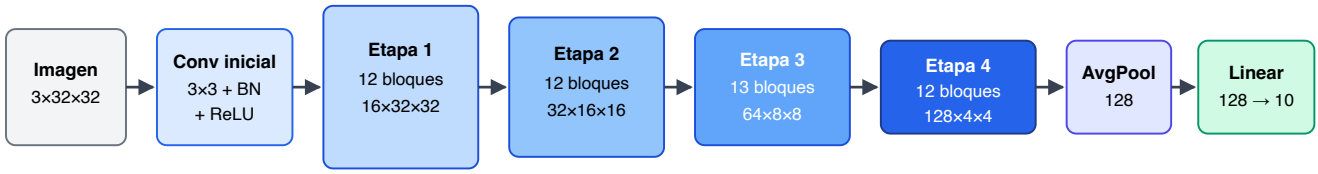


Figura 3. Flujo de datos a través de la ResNet-100 (entrada de 32×32, ancho base 16, 10 clases).

Sección	Bloques	Canales	Salida (C×H×W)	Capas con pesos	Parámetros
Convolución inicial + BN	—	3 → 16	16×32×32	1	464
Etapa 1	12	16	16×32×32	24	56.064
Etapa 2	12	32	32×16×16	24	218.112
Etapa 3	13	64	64×8×8	26	943.360
Etapa 4	12	128	128×4×4	24	3.471.360
AdaptiveAvgPool2d	—	128	128×1×1	0	0
Clasificador lineal	—	128 → 10	10	1	1.290
Total	**49**			**100**	**4.690.650**

CÁLCULO DE PARÁMETROS

$$P_{\text{conv}} = k^2\, C_{in}\, C_{out}, \qquad P_{\text{BN}} = 2\,C, \qquad P_{\text{lineal}} = 128\,K + K$$

Ejemplos con los valores de la tabla:

- Convolución inicial: $3^2 \cdot 3 \cdot 16 + 2 \cdot 16 = 432 + 32 = 464$.
- Clasificador: $128 \cdot 10 + 10 = 1\,290$.
- Etapa 4 (el primer bloque recibe 64 canales; los otros 11, 128):

$$\underbrace{(9 \cdot 64 \cdot 128 + 256) + (9 \cdot 128 \cdot 128 + 256)}_{\text{primer bloque}} + \underbrace{11 \cdot 2\,(9 \cdot 128 \cdot 128 + 256)}_{\text{11 bloques restantes}} = 3\,471\,360$$

Como P_{conv} depende del producto $C_{in}C_{out}$ y ambos son proporcionales a a, el total crece aproximadamente como a^2: duplicar `ancho_base` multiplica los parámetros por casi 4.

3.4 Conteo de las 100 capas

Se consideran «capas» las que tienen pesos entrenables principales: convoluciones (`Conv2d`) y capas totalmente conectadas (`Linear`). Es la convención habitual en la literatura (por ejemplo, ResNet-50 o ResNet-101). BatchNorm, ReLU y pooling no se cuentan. Con b_e bloques en la etapa e, definidos en `BLOQUES_POR_ETAPA = (12, 12, 13, 12)`:

NÚMERO DE CAPAS

$$L = \underbrace{1}_{\text{conv inicial}} + 2\sum_{e=1}^{4} b_e + \underbrace{1}_{\text{lineal}} = 1 + 2\,(12 + 12 + 13 + 12) + 1 = 1 + 98 + 1 = 100$$

La función `contar_capas(modelo)` recorre todos los módulos y cuenta los que son `Conv2d` o `Linear`; la prueba automatizada `test_la_red_tiene_100_capas` y el endpoint `GET /api/modelo/` verifican que el resultado sea exactamente 100.

3.5 Atajos sin parámetros

Cuando un bloque cambia de etapa, su entrada y su salida tienen dimensiones distintas (el doble de canales y la mitad de resolución), por lo que no se pueden sumar directamente. La clase `AtajoSinParametros` resuelve esto con la llamada «opción A» del artículo original de ResNet: submuestrea tomando uno de cada dos píxeles (`x[:, :, ::2, ::2]`) y añade canales con ceros (`F.pad`) hasta igualar los de la salida.

ATAJO ENTRE ETAPAS (ÍNDICES DESDE 0)

$$\mathcal{S}(\mathbf{x})_{c,\,i,\,j} = \begin{cases} x_{c,\,2i,\,2j} & \text{si } 0 \le c < C_{in} \\ 0 & \text{si } C_{in} \le c < 2\,C_{in} \end{cases}$$

Como este atajo no tiene pesos, no añade capas al conteo y el total se mantiene en exactamente 100. Dentro de una misma etapa el atajo es simplemente la identidad (`nn.Identity`).

3.6 Inicialización de pesos

El método `_inicializar_pesos` prepara la red para entrenar de forma estable:

VALORES INICIALES

$$W_{\text{conv}} \sim \mathcal{N}\left(0,\ \frac{2}{\text{fan}_{out}}\right), \quad \text{fan}_{out} = k^2\, C_{out} \qquad \gamma = 1,\ \ \beta = 0 \qquad W_{\text{fc}} \sim \mathcal{N}\left(0,\ 0{,}01^2\right),\ \ \mathbf{b} = \mathbf{0}$$

La inicialización de Kaiming (He) para las convoluciones mantiene estable la varianza de las activaciones a través de las capas con ReLU. $\mathcal{N}(0, \sigma^2)$ denota una distribución normal de media 0 y varianza σ^2.

Además, la escala γ_2 de la **última BatchNorm de cada bloque se inicializa en cero** (*zero-init residual*). Así, al comenzar el entrenamiento:

$$\gamma_2 = 0 \implies \mathcal{F}(\mathbf{x}) = \mathbf{0} \implies \mathbf{y} = \text{ReLU}\big(\mathcal{S}(\mathbf{x})\big) = \mathcal{S}(\mathbf{x})$$

La última igualdad se cumple porque $\mathbf{x}$ proviene de una ReLU y ya es no negativo. Cada bloque empieza comportándose como la identidad: la red de 100 capas arranca como si fuera poco profunda y va «activando» sus bloques a medida que aprende, lo que evita inestabilidades al principio del entrenamiento.

4. Datos y preprocesamiento

El módulo `cnn/data.py` prepara las imágenes para la red. La función `crear_dataloaders()` devuelve el cargador de entrenamiento, el de validación y la lista de clases.

4.1 Fuentes de datos admitidas

CIFAR-10 (`--dataset cifar10`, por defecto)

Conjunto público de 60.000 imágenes a color de 32×32 píxeles: 50.000 para entrenamiento y 10.000 para validación. Se descarga automáticamente la primera vez en `data/` (unos 170 MB). Sus 10 clases se traducen al español en `cnn/config.py`:

Índice	Clase	Índice	Clase
0	avion	5	perro
1	automovil	6	rana
2	pajaro	7	caballo
3	gato	8	barco
4	venado	9	camion

Imágenes propias (`--dataset carpeta`)

Se utiliza el formato `ImageFolder` de torchvision: cada subcarpeta es una clase y su nombre se convierte en la etiqueta. Debe existir una partición de entrenamiento y otra de validación con las mismas clases:

```
mis_imagenes/
├── train/
│   ├── perro/   img001.jpg, img002.jpg, ...
│   ├── gato/    ...
│   └── conejo/  ...
└── val/
    ├── perro/   ...
    ├── gato/    ...
    └── conejo/  ...
```

Las clases se ordenan alfabéticamente. El número de clases detectado define el tamaño K de la capa de salida de la red, y la lista se guarda en el checkpoint para que la API devuelva los nombres correctos.

4.2 Transformaciones

Entrenamiento (con aumento de datos)

1. `Resize(tamano)`: lado menor a `tamano` píxeles.
2. `RandomCrop(tamano, padding=p)`: rellena p píxeles por lado y recorta una región aleatoria.

Evaluación e inferencia

1. `Resize(tamano)`.
2. `CenterCrop(tamano)`: recorte cuadrado central.
3. `ToTensor()`.
4. `Normalize(media, desviacion)`.

3. `RandomHorizontalFlip()`: espejo horizontal con probabilidad 0,5.
4. `ToTensor()`: convierte a tensor con valores en $[0, 1]$.
5. `Normalize(media, desviacion)`.

Sin aleatoriedad: la misma imagen siempre produce el mismo resultado.

RELLENO DEL RECORTE ALEATORIO Y NORMALIZACIÓN

$$p = \left\lfloor \frac{\texttt{tamano}}{8} \right\rfloor \qquad x_c = \frac{v_c}{255}, \qquad x'_c = \frac{x_c - \mu_c}{\sigma_c}, \quad c \in \{R, G, B\}$$

$$\boldsymbol{\mu} = (0{,}4914,\ 0{,}4822,\ 0{,}4465), \qquad \boldsymbol{\sigma} = (0{,}2470,\ 0{,}2435,\ 0{,}2616)$$

v_c es el valor original del píxel (0–255). Para `tamano` = 32 el relleno es $p = 4$; para 64, $p = 8$. $\boldsymbol{\mu}$ y $\boldsymbol{\sigma}$ son la media y la desviación estándar de CIFAR-10 por canal; tras normalizar, cada canal queda aproximadamente con media 0 y desviación 1.

El aumento de datos genera variaciones de cada imagen en cada época, lo que reduce el sobreajuste. Los valores de normalización se guardan en el checkpoint, de modo que la inferencia aplica exactamente la misma transformación que el entrenamiento.

Cualquier imagen de entrada, sin importar su tamaño, formato (JPEG, PNG, etc.) o modo de color, se convierte a RGB y se redimensiona al tamaño con el que se entrenó el modelo antes de entrar en la red.

5. Entrenamiento

El script `cnn/train.py` se ejecuta como módulo (`python -m cnn.train`) desde la raíz del proyecto. Sigue el ciclo clásico de aprendizaje supervisado.

5.1 Algoritmo

1. Selecciona el dispositivo de cómputo: CUDA (GPU NVIDIA), MPS (GPU de Apple Silicon) o CPU, en ese orden de preferencia.
2. Crea los cargadores de datos y construye la ResNet-100 con tantas salidas como clases.
3. Muestra el número de capas, de parámetros y el dispositivo.
4. Para cada época:
   a. **Entrenamiento:** para cada lote, calcula la predicción, mide la pérdida de entropía cruzada, propaga el gradiente hacia atrás (*backpropagation*) y actualiza los pesos.
   b. **Validación:** evalúa el modelo sobre los datos de validación sin calcular gradientes (`torch.inference_mode`).
   c. Ajusta la tasa de aprendizaje con el planificador.
   d. Imprime pérdida y precisión de entrenamiento y validación, y la duración de la época.
   e. Si la precisión de validación supera la mejor obtenida hasta ahora, guarda el checkpoint.
5. Al terminar, muestra la mejor precisión de validación alcanzada.

5.2 Formulación matemática

FUNCIÓN DE PÉRDIDA: ENTROPÍA CRUZADA

$$\mathcal{L}(\theta) \;=\; -\frac{1}{N}\sum_{n=1}^{N} \log p_{n,\,y_n}, \qquad p_{n,k} = \operatorname{softmax}(\mathbf{z}_n)_k$$

N es el tamaño del lote, y_n la clase correcta de la imagen n y p_{n,y_n} la probabilidad que el modelo le asigna. La pérdida es 0 si el modelo asigna probabilidad 1 a la clase correcta y crece sin límite cuando esa probabilidad se acerca a 0. θ representa todos los pesos de la red.

OPTIMIZADOR: SGD CON MOMENTO DE NESTEROV Y WEIGHT DECAY

$$\begin{aligned} \mathbf{g}_t &= \nabla_\theta \,\mathcal{L}(\theta_{t-1}) + \lambda\,\theta_{t-1} \\ \mathbf{b}_t &= \mu\,\mathbf{b}_{t-1} + \mathbf{g}_t \qquad\qquad \mu = 0{,}9, \quad \lambda = 5\times10^{-4} \\ \theta_t &= \theta_{t-1} - \eta\,\left(\mathbf{g}_t + \mu\,\mathbf{b}_t\right) \end{aligned}$$

$\mathbf{g}_t$ es el gradiente más el término de *weight decay* $\lambda\theta$ (regularización L2), $\mathbf{b}_t$ acumula la «velocidad» (momento μ) y η es la tasa de aprendizaje vigente. Es la forma exacta que implementa `torch.optim.SGD` con `nesterov=True`; se aplica una vez por lote.

PLANIFICADOR: TASA DE APRENDIZAJE CON RECOCIDO COSENO

$$\eta_t \;=\; \eta_{\min} + \frac{1}{2}\left(\eta_0 - \eta_{\min}\right)\left(1 + \cos\frac{\pi\, t}{T}\right), \qquad \eta_0 = 0{,}1, \quad \eta_{\min} = 0$$

t es el número de épocas completadas y T el total (`--epocas`). La tasa empieza en η_0, vale $\eta_0/2$ a mitad del entrenamiento y termina cerca de 0. Por ejemplo, con $T = 30$: $\eta_0 = 0{,}1$, $\eta_{15} = 0{,}05$ y $\eta_{29} \approx 0{,}00027$. Pasos grandes al principio exploran; pasos pequeños al final afinan.

MÉTRICA: PRECISIÓN (ACCURACY)

$$\text{precisión} \;=\; \frac{1}{N}\sum_{n=1}^{N} 1\big[\,\hat{y}_n = y_n\,\big], \qquad \hat{y}_n = \arg\max_k \; z_{n,k}$$

$1[\cdot]$ vale 1 si la condición se cumple y 0 si no. La pérdida y la precisión de cada época se promedian sobre todas las imágenes procesadas, ponderando cada lote por su tamaño.

5.3 Hiperparámetros

Elemento	Configuración
Función de pérdida	`CrossEntropyLoss` .
Optimizador	SGD, momento $\mu = 0{,}9$, Nesterov, *weight decay* $\lambda = 5 \times 10^{-4}$.
Tasa de aprendizaje inicial	$\eta_0 = 0{,}1$ (configurable con `--lr`).
Planificador	`CosineAnnealingLR` con T = número de épocas.
Tamaño de lote	$N = 128$ imágenes (configurable con `--lote`).
Épocas	$T = 30$ por defecto (configurable con `--epocas`).

5.4 Opciones de línea de comandos

Opción	Por defecto	Descripción
`--dataset`	`cifar10`	`cifar10` o `carpeta`.
`--ruta-datos`	`data/`	Carpeta de los datos. Obligatoria con `--dataset carpeta`.
`--tamano-imagen`	32	Lado en píxeles al que se redimensionan las imágenes.
`--ancho-base`	16	Canales de la primera etapa (16, 32, 64, 128 por etapa).
`--epocas`	30	Número de pasadas completas sobre los datos.
`--lote`	128	Imágenes por lote.
`--lr`	0.1	Tasa de aprendizaje inicial.
`--trabajadores`	2	Procesos paralelos para cargar imágenes (0 = en el proceso principal).
`--dispositivo`	`auto`	`auto`, `cpu`, `cuda` o `mps`.
`--salida`	`checkpoints/resnet100.pt`	Ruta donde se guarda el mejor modelo.
`--limite-lotes`	sin límite	Procesa solo N lotes por época, tanto en entrenamiento como en validación. Útil para pruebas rápidas.

5.5 Salida en consola

```
Modelo ResNet-100 | capas: 100 | parámetros: 4,690,650 | dispositivo: mps
Época 001/030 | pérdida ent 1.6342 prec ent 39.12% | pérdida val 1.3518 prec val 51.04% |
58.3s
  -> checkpoint guardado en checkpoints/resnet100.pt
Época 002/030 | ...
Mejor precisión de validación: 72.06%
```

(Los valores por época son ilustrativos; dependen del hardware y de la ejecución.)

5.6 Contenido del checkpoint

El archivo `.pt` es un diccionario serializado con `torch.save`. Además de los pesos, guarda todo lo necesario para reconstruir el modelo y preprocesar imágenes igual que en el entrenamiento:

Clave	Contenido
`estado_modelo`	Pesos de todas las capas (`state_dict`).
`clases`	Lista ordenada de nombres de clase.
`tamano_imagen`	Tamaño de entrada usado al entrenar.
`ancho_base`	Ancho base de la red.
`media` , `desviacion`	Valores de normalización μ y σ.
`precision_validacion`	Precisión de validación del modelo guardado.

6. Inferencia (predicción)

6.1 La clase `Predictor`

`cnn/predictor.py` encapsula todo lo necesario para clasificar una imagen. Es usado tanto por la API como por el script de predicción local.

Carga

- Si el checkpoint existe, lee las clases, el tamaño de imagen, el ancho base y la normalización; construye la red con esos parámetros y carga los pesos. `modelo_entrenado` vale `True`.
- Si no existe, registra una advertencia y crea una red con **pesos aleatorios** y las clases de CIFAR-10. Así la API puede arrancar y probarse sin entrenar, pero las predicciones no tienen significado y `modelo_entrenado` vale `False`.
- Pone el modelo en modo evaluación (`eval()`), lo que hace que BatchNorm use sus promedios móviles.

Predicción: `predecir(imagen, top_k=3)`

1. Abre la imagen. Acepta un objeto `PIL.Image`, `bytes`, una ruta (`str` o `Path`) o un archivo abierto.
2. La convierte a RGB y le aplica las transformaciones de evaluación.
3. Añade la dimensión de lote y la pasa por la red sin calcular gradientes, obteniendo los logits $\mathbf{z}$.
4. Aplica softmax para obtener probabilidades.
5. Selecciona las clases más probables.

SOFTMAX, CONFIANZA Y TOP-K

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_{k=1}^{K} p_k = 1, \qquad \text{confianza} = \max_k \; p_k$$

$$k_{\text{efectivo}} = \max\left(1,\; \min(\texttt{top_k},\; K)\right)$$

El resultado contiene las k_{efectivo} clases con mayor p_k, ordenadas de mayor a menor. Por ejemplo, pedir `top_k = 20` con 10 clases devuelve las 10.

```
{
  "clase": "barco",
  "confianza": 0.975416,
  "top_k": [
    {"clase": "barco", "probabilidad": 0.975416},
    {"clase": "camion", "probabilidad": 0.020401},
    {"clase": "avion", "probabilidad": 0.00352}
  ],
  "modelo_entrenado": true
}
```

Información: `informacion()`

Devuelve la arquitectura, el número de capas y parámetros, las clases, el tamaño de imagen, el dispositivo, si el modelo está entrenado y la ruta del checkpoint.

6.2 Instancia compartida

Cargar la red es costoso, así que la función `obtener_predictor()` crea el `Predictor` una sola vez por proceso y lo reutiliza en todas las peticiones. Usa un *lock* con doble comprobación para que, si llegan varias peticiones simultáneas al arrancar, solo una de ellas cargue el modelo.

> **Importante:** como el modelo se carga una única vez, después de reentrenar o de sustituir el checkpoint hay que **reiniciar el servidor de la API** para que use los nuevos pesos.

6.3 Predicción local sin API

El módulo `cnn/predict.py` clasifica imágenes directamente desde la terminal:

```
python -m cnn.predict foto1.jpg foto2.png --top-k 5
python -m cnn.predict foto.jpg --checkpoint otra_ruta/modelo.pt
```

Imprime un JSON por imagen con el campo `imagen` más el resultado de `predecir()`.

7. API REST con Django REST Framework

La carpeta `api/` contiene un proyecto Django (`config`) con una aplicación (`procesamiento`). Todas las rutas de la API cuelgan de `/api/` y el panel de administración está en `/admin/`.

7.1 Configuración (`api/config/settings.py`)

- Añade la raíz del proyecto a `sys.path` para poder importar el paquete `cnn`.
- Base de datos SQLite en `api/db.sqlite3`.
- Idioma `es` y zona horaria `America/Mexico_City`.
- Archivos subidos en `api/media/`, servidos en `/media/` solo cuando `DEBUG` está activo.
- DRF acepta `multipart/form-data`, formularios y JSON; pagina los listados de 20 en 20.

Variables de entorno

Variable	Por defecto	Uso
`CNN_CHECKPOINT`	`checkpoints/resnet100.pt`	Ruta del modelo entrenado que carga la API.
`CNN_TAMANO_MAXIMO_IMAGEN_MB`	`10`	Tamaño máximo permitido por imagen.
`DJANGO_SECRET_KEY`	clave de desarrollo	Clave criptográfica de Django. Debe cambiarse en producción.
`DJANGO_DEBUG`	`1`	Modo depuración. Usar `0` en producción.
`DJANGO_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Hosts desde los que se acepta tráfico, separados por coma.

7.2 Modelo de datos

El modelo `ImagenProcesada` registra cada imagen subida y su clasificación:

Campo	Tipo	Descripción
`id`	BigAutoField	Identificador autoincremental.
`imagen`	ImageField	Archivo guardado en `media/imagenes/AAAA/MM/DD/`.
`clase`	CharField(100)	Clase más probable.
`confianza`	FloatField	Probabilidad de la clase más probable (0 a 1).
`resultados`	JSONField	Lista top-k de `{clase, probabilidad}`.
`modelo_entrenado`	BooleanField	Indica si se usó un checkpoint entrenado.
`creado`	DateTimeField	Fecha de creación (automática). Los listados se ordenan del más reciente al más antiguo.

7.3 Validaciones

- **Formato:** `ImageField` verifica con Pillow que el archivo sea realmente una imagen; si no lo es, responde `400`.
- **Tamaño:** se rechaza toda imagen mayor que `CNN_TAMANO_MAXIMO_IMAGEN_MB` × 1024 × 1024 bytes.
- **top_k:** entero entre 1 y 20; vale 3 si no se envía.
- **Lote:** el endpoint de predicciones admite entre 1 y 32 imágenes por petición.

7.4 Flujo de una petición

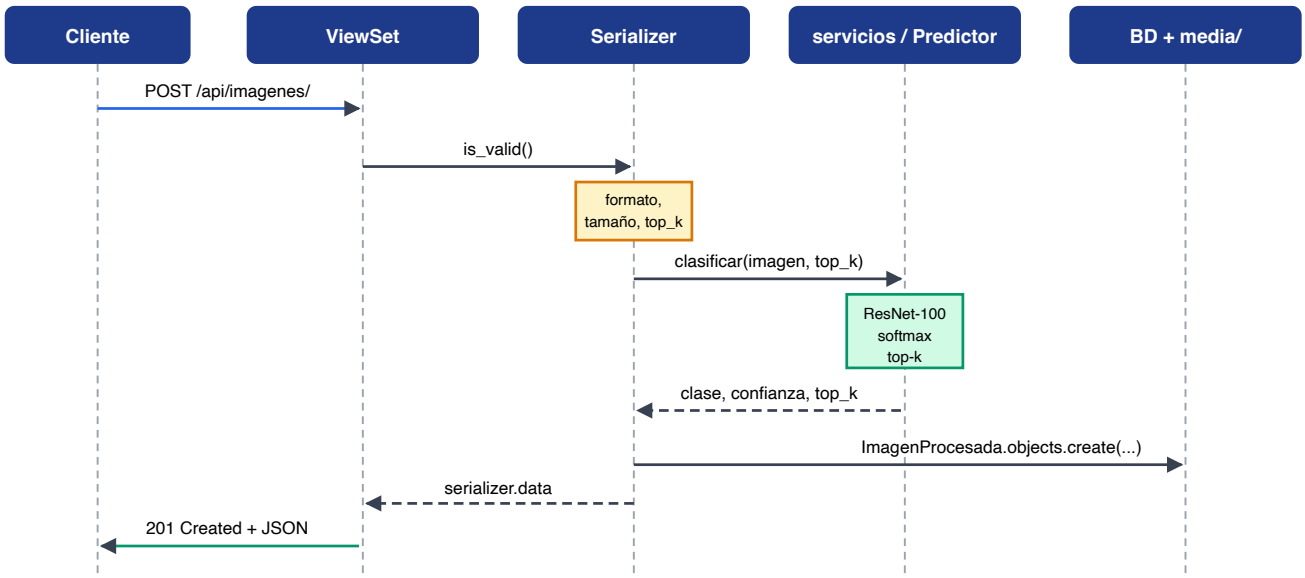


Figura 4. Secuencia de procesamiento de `POST /api/imagenes/`.

La función `servicios.clasificar()` rebobina el archivo subido (`seek(0)`) antes y después de la inferencia. Esto es necesario porque la validación y la lectura de la imagen avanzan el puntero del archivo; sin rebobinar, Django guardaría un archivo vacío o la red recibiría datos incompletos.

7.5 Referencia de endpoints

Método	Ruta	Descripción	Respuesta
GET	`/api/`	Raíz navegable del enrutador de DRF.	200
GET	`/api/modelo/`	Información de la red cargada.	200
POST	`/api/imagenes/`	Sube, clasifica y guarda una imagen.	201 / 400
GET	`/api/imagenes/`	Lista paginada de imágenes procesadas.	200
GET	`/api/imagenes/{id}/`	Detalle de una imagen procesada.	200 / 404
DELETE	`/api/imagenes/{id}/`	Elimina el registro y el archivo de imagen.	204 / 404
POST	`/api/predicciones/`	Clasifica de 1 a 32 imágenes sin guardarlas.	200 / 400

GET /api/modelo/

```
# Petición
curl http://127.0.0.1:8000/api/modelo/

# Respuesta 200
{
  "arquitectura": "ResNet-100",
  "capas": 100,
  "parametros": 4690650,
  "clases": ["avion", "automovil", "pajaro", "gato", "venado",
             "perro", "rana", "caballo", "barco", "camion"],
  "tamano_imagen": 32,
  "dispositivo": "mps",
  "modelo_entrenado": true,
  "checkpoint": ".../checkpoints/resnet100.pt"
}
```

POST /api/imagenes/

Cuerpo `multipart/form-data`: `imagen` (archivo, obligatorio) y `top_k` (entero, opcional).

```
# Petición
curl -F "imagen=@barco.jpg" -F "top_k=3" http://127.0.0.1:8000/api/imagenes/

# Respuesta 201
{
  "id": 1,
  "imagen": "http://127.0.0.1:8000/media/imagenes/2026/10/03/barco.jpg",
  "clase": "barco",
  "confianza": 0.975416,
  "resultados": [
    {"clase": "barco", "probabilidad": 0.975416},
    {"clase": "camion", "probabilidad": 0.020401},
    {"clase": "avion", "probabilidad": 0.00352}
  ],
  "modelo_entrenado": true,
  "creado": "2026-10-03T23:37:30-06:00"
}
```

GET /api/imagenes/?page=1

```
{
  "count": 2,
  "next": null,
  "previous": null,
  "results": [ { "id": 2, "clase": "caballo", ... }, { "id": 1, "clase": "barco", ... }
]
}
```

POST `/api/predicciones/`

Cuerpo `multipart/form-data`: el campo `imagenes` repetido una vez por archivo y `top_k` opcional. No guarda nada en la base de datos.

```
# Petición
curl -F "imagenes=@a.jpg" -F "imagenes=@b.png" http://127.0.0.1:8000/api/predicciones/

# Respuesta 200
{
  "resultados": [
    {"nombre": "a.jpg", "clase": "gato", "confianza": 0.81, "top_k": [...],
"modelo_entrenado": true},
    {"nombre": "b.png", "clase": "perro", "confianza": 0.66, "top_k": [...],
"modelo_entrenado": true}
  ]
}
```

> Diferencia de nombres: en `/api/imagenes/` la lista de probabilidades se llama `resultados` (campo del modelo), mientras que en `/api/predicciones/` cada elemento la devuelve como `top_k` (salida directa del predictor).

DELETE `/api/imagenes/{id}/`

```
curl -X DELETE http://127.0.0.1:8000/api/imagenes/1/     # 204 No Content
```

Errores de validación (400)

```
{ "imagen": ["Adjunte una imagen válida. El archivo adjunto o bien no es una imagen o bien
está dañado."] }
{ "imagen": ["La imagen supera el límite de 10 MB."] }
{ "top_k": ["Asegúrese de que este valor es menor o igual a 20."] }
```

7.6 Panel de administración

`ImagenProcesada` está registrada en el admin de Django (`/admin/`), con columnas de id, imagen, clase, confianza, estado del modelo y fecha, y filtros por clase y por `modelo_entrenado`. Los campos del resultado son de solo lectura. Para entrar se necesita un superusuario (ver procedimiento 9.6).

8. Cliente de la API

`cliente/cliente.py` ofrece dos formas de consumir la API: una interfaz de línea de comandos y la clase `ClienteCNN` para usar desde otros programas Python. La URL base se toma de `--url`, de la variable `CNN_API_URL` o, por defecto, `http://127.0.0.1:8000/api`.

8.1 Línea de comandos

Comando	Acción	Endpoint
`info`	Muestra la información del modelo.	`GET modelo/`
`procesar IMAGEN [--top-k N]`	Clasifica y guarda una imagen.	`POST imagenes/`
`predecir IMG1 IMG2 ... [--top-k N]`	Clasifica varias imágenes sin guardarlas.	`POST predicciones/`
`listar [--pagina N]`	Lista las imágenes procesadas.	`GET imagenes/`
`obtener ID`	Muestra el detalle de una imagen.	`GET imagenes/ID/`
`eliminar ID`	Elimina una imagen procesada.	`DELETE imagenes/ID/`

```
python cliente/cliente.py info
python cliente/cliente.py procesar foto.jpg --top-k 5
python cliente/cliente.py predecir foto1.jpg foto2.png
python cliente/cliente.py listar --pagina 2
python cliente/cliente.py obtener 1
python cliente/cliente.py eliminar 1
python cliente/cliente.py --url http://servidor:8000/api info
```

Las respuestas se imprimen como JSON con formato. Si ocurre un error HTTP, de red o de lectura de archivo, el mensaje se escribe en la salida de error y el programa termina con código 1.

8.2 Uso como librería

```python
from cliente.cliente import ClienteCNN

cliente = ClienteCNN("http://127.0.0.1:8000/api", timeout=60)

info = cliente.informacion_modelo()
resultado = cliente.procesar("foto.jpg", top_k=3)
print(resultado["clase"], resultado["confianza"])

lote = cliente.predecir(["a.jpg", "b.png"])
pagina = cliente.listar(pagina=1)
detalle = cliente.obtener(resultado["id"])
cliente.eliminar(resultado["id"])
```

La clase reutiliza una sesión HTTP (`requests.Session`), deduce el tipo MIME de cada archivo por su extensión, cierra los archivos automáticamente y lanza `RuntimeError` con el código y el detalle cuando la API responde con error.

9. Procedimientos paso a paso

Todos los comandos se ejecutan desde la raíz del proyecto, salvo que se indique lo contrario.

9.1 Instalación del entorno

1. Crear el entorno virtual de Python:

```
python3 -m venv .venv
```

2. Activarlo (repetir en cada terminal nueva):

```
source .venv/bin/activate        # macOS / Linux
.venv\Scripts\activate           # Windows
```

3. Instalar las dependencias:

```
pip install -r requirements.txt
```

4. Comprobar la instalación ejecutando las pruebas de la red:

```
pytest
```

9.2 Entrenar con CIFAR-10

1. Hacer primero una prueba rápida para validar que todo funciona (descarga CIFAR-10 y procesa solo 5 lotes por época):

```
python -m cnn.train --epocas 1 --limite-lotes 5 --salida checkpoints/prueba.pt
```

2. Lanzar el entrenamiento completo:

```
python -m cnn.train --epocas 50
```

3. Observar la precisión de validación en cada época. El mejor modelo se guarda automáticamente en `checkpoints/resnet100.pt`, de modo que se puede interrumpir con `Ctrl+C` y conservar el mejor resultado obtenido hasta ese momento.

> Con más épocas la precisión suele mejorar: con la configuración por defecto, una ResNet de este tipo puede superar el 90 % en CIFAR-10 tras unas 100–200 épocas. El tiempo por época depende mucho del hardware (GPU frente a CPU).

9.3 Entrenar con imágenes propias

1. Organizar las imágenes en `train/<clase>/` y `val/<clase>/` (sección 4.1). Se recomienda reservar entre el 10 % y el 20 % de las imágenes de cada clase para validación.

2. Entrenar indicando la carpeta y, si las imágenes tienen más detalle, un tamaño mayor:

```
python -m cnn.train --dataset carpeta --ruta-datos mis_imagenes --tamano-imagen 64 --
epocas 30
```

3. Con pocos datos o poca memoria, reducir el lote (por ejemplo `--lote 32`). Para conservar el modelo de CIFAR-10, guardar en otra ruta con `--salida checkpoints/mi_modelo.pt`.

9.4 Probar el modelo sin la API

```
python -m cnn.predict foto.jpg --top-k 5
python -m cnn.predict foto.jpg --checkpoint checkpoints/mi_modelo.pt
```

9.5 Poner en marcha la API

1. Entrar a la carpeta de la API y crear las tablas de la base de datos (solo la primera vez o tras cambios en los modelos):

```
cd api
python manage.py migrate
```

2. Iniciar el servidor de desarrollo:

```
python manage.py runserver                # http://127.0.0.1:8000
python manage.py runserver 0.0.0.0:8000   # accesible desde la red local
```

3. Para usar un checkpoint distinto del predeterminado:

```
CNN_CHECKPOINT=../checkpoints/mi_modelo.pt python manage.py runserver
```

4. Verificar que responde y que el modelo está entrenado (`"modelo_entrenado": true`), desde otra terminal o en el navegador, donde DRF muestra una interfaz navegable:

```
curl http://127.0.0.1:8000/api/modelo/
```

9.6 Acceder al panel de administración

```
cd api
python manage.py createsuperuser     # pide usuario, correo y contraseña
python manage.py runserver
```

Después, abrir `http://127.0.0.1:8000/admin/` e iniciar sesión.

9.7 Clasificar imágenes

Con el servidor en marcha, en otra terminal con el entorno virtual activo:

```
python cliente/cliente.py procesar foto.jpg --top-k 3        # guarda el resultado
python cliente/cliente.py predecir foto1.jpg foto2.png       # sin guardar
python cliente/cliente.py listar                             # historial
```

9.8 Actualizar el modelo en la API

1. Entrenar un nuevo modelo (sección 9.2 o 9.3).
2. Detener el servidor (`Ctrl`+`C`) y volver a iniciarlo, para que cargue el nuevo checkpoint.
3. Comprobar con `GET /api/modelo/` que las clases y el estado son los esperados. Los registros guardados antes conservan la clasificación del modelo anterior.

9.9 Ejecutar las pruebas

```
pytest                                  # red neuronal (desde la raíz)
cd api && python manage.py test          # API
```

9.10 Recomendaciones para producción

- Definir `DJANGO_SECRET_KEY` con un valor secreto y aleatorio, `DJANGO_DEBUG=0` y `DJANGO_ALLOWED_HOSTS` con el dominio real.
- Usar un servidor WSGI (por ejemplo Gunicorn con `config.wsgi`) en lugar de `runserver`.
- Servir `/media/` con un servidor web (Nginx, almacenamiento de objetos, etc.), ya que Django solo lo hace con `DEBUG` activo.
- Añadir autenticación y límites de peticiones en DRF (ver sección 13).
- Cada proceso del servidor carga su propia copia del modelo; dimensionar la memoria según el número de procesos.

10. Pruebas automatizadas

El proyecto incluye dos suites. Ninguna requiere un modelo entrenado ni conexión a internet: las pruebas generan imágenes sintéticas con Pillow y, cuando hace falta, usan pesos aleatorios.

10.1 Red neuronal — `tests/test_model.py` (pytest)

Prueba	Qué verifica
`test_la_red_tiene_100_capas`	Que `contar_capas(ResNet100())` sea exactamente 100.
`test_forward_produce_logits_por_clase`	Con entradas de 32×32 y 64×64 y 7 clases, la salida tiene forma `(2, 7)` : la red acepta distintos tamaños y número de clases.
`test_predictor_sin_checkpoint_clasifica_bytes`	Sin checkpoint, el predictor clasifica una imagen JPEG en bytes, devuelve `modelo_entrenado = False` , 3 resultados coherentes y una confianza entre 0 y 1.

`pytest.ini` define `tests` como carpeta de pruebas y añade la raíz al `pythonpath` .

10.2 API — `api/procesamiento/tests.py` (Django)

Prueba	Qué verifica
`test_info_modelo_reporta_100_capas`	`GET /api/modelo/` responde 200 y reporta 100 capas.
`test_subir_imagen_la_procesa_y_guarda`	`POST /api/imagenes/` con `top_k=5` responde 201, devuelve 5 resultados, guarda un registro y su detalle es accesible.
`test_rechaza_archivo_que_no_es_imagen`	Un archivo de texto recibe 400 y no se guarda nada.
`test_prediccion_por_lote_sin_guardar`	`POST /api/predicciones/` con 2 imágenes devuelve sus resultados en orden y no crea registros.
`test_eliminar_imagen`	`DELETE` responde 204 y elimina el registro.

Las pruebas de la API redirigen `MEDIA_ROOT` a una carpeta temporal que se borra al terminar, y Django crea una base de datos de prueba independiente, por lo que no alteran los datos reales.

11. Estado actual del proyecto

> El proyecto está operativo de punta a punta: la red está implementada y entrenada, la API funciona con el modelo entrenado y ya registra imágenes procesadas.

Elemento	Estado
Checkpoint	`checkpoints/resnet100.pt` (≈ 19 MB), entrenado con CIFAR-10.
Precisión de validación	**72,06 %** en el conjunto de validación de CIFAR-10.
Configuración del modelo	10 clases, imágenes de 32×32, ancho base 16, 100 capas, 4.690.650 parámetros.
Datos	CIFAR-10 descargado en `data/`.
Base de datos	Migración inicial aplicada; 2 imágenes de prueba procesadas (un diagrama y una ilustración), clasificadas como *barco* (97,5 %) y *caballo* (89,2 %). Como no pertenecen a ninguna de las 10 clases, esas clasificaciones no son representativas.
Dispositivo detectado	MPS (GPU de Apple Silicon) en el equipo de desarrollo.

Una precisión del 72 % significa que el modelo acierta la clase de aproximadamente 7 de cada 10 imágenes que nunca ha visto. Como referencia, un clasificador que eligiera al azar entre 10 clases acertaría $1/K = 10\,\%$ de las veces. Para una red de este tamaño es un resultado de entrenamiento corto; entrenar más épocas (procedimiento 9.2) debería mejorarlo de forma considerable.

12. Solución de problemas

Síntoma	Causa probable	Solución
La API responde `"modelo_entrenado": false` y las predicciones parecen aleatorias.	No se encontró el checkpoint.	Entrenar el modelo, o definir `CNN_CHECKPOINT` con la ruta correcta y reiniciar el servidor.
Tras reentrenar, la API sigue dando los resultados anteriores.	El modelo se carga una vez por proceso.	Reiniciar el servidor.
`ModuleNotFoundError: No module named 'cnn'`	El script se ejecutó desde otra carpeta o como archivo.	Ejecutar desde la raíz con `python -m cnn.train` / `python -m cnn.predict`.
Errores o bloqueos del `DataLoader` (sobre todo en macOS/Windows).	Problemas con procesos paralelos de carga.	Usar `--trabajadores 0`.
Falta de memoria (*out of memory*) en GPU o MPS.	Lote o tamaño de imagen demasiado grandes.	Reducir `--lote` o `--tamano-imagen`, o usar `--dispositivo cpu`.
Error con una operación no soportada en MPS.	Soporte parcial de PyTorch en Apple Silicon.	Usar `--dispositivo cpu` o definir `PYTORCH_ENABLE_MPS_FALLBACK=1`.
La pérdida se vuelve `nan` o la precisión no mejora.	Tasa de aprendizaje demasiado alta para la configuración.	Reducir `--lr` (por ejemplo a 0.05 o 0.01).
Falla la descarga de CIFAR-10.	Sin conexión o descarga interrumpida.	Revisar la conexión, borrar el archivo parcial de `data/` y repetir.
`400`: «La imagen supera el límite de 10 MB».	Archivo mayor al permitido.	Reducir la imagen o aumentar `CNN_TAMANO_MAXIMO_IMAGEN_MB`.
`400`: «Adjunte una imagen válida».	El archivo no es una imagen o está dañado.	Enviar un archivo JPEG, PNG u otro formato soportado por Pillow.
`400 Bad Request` / `DisallowedHost` al acceder desde otro equipo.	El host no está en `ALLOWED_HOSTS`.	Añadirlo a `DJANGO_ALLOWED_HOSTS`.
El cliente muestra *Connection refused*.	El servidor no está en marcha o la URL es incorrecta.	Iniciar `runserver` o indicar `--url` / `CNN_API_URL`.
`no such table: procesamiento_imagenprocesada`	No se aplicaron las migraciones.	Ejecutar `python manage.py migrate` en `api/`.
Las URL de `/media/` dan 404 en producción.	Django solo sirve archivos subidos con `DEBUG=1`.	Configurar un servidor web para `api/media/`.

13. Limitaciones y posibles mejoras

13.1 Limitaciones actuales

- **Siempre elige una clase conocida:** como softmax reparte toda la probabilidad entre las K clases ($\sum_k p_k = 1$), el modelo asigna toda imagen a una de ellas aunque no tenga relación (por ejemplo, un diagrama puede clasificarse como *barco* con alta confianza). No hay una categoría de «desconocido».
- **Sin autenticación:** la API usa los permisos por defecto de DRF, por lo que cualquiera con acceso a la red puede subir, ver o borrar imágenes.
- **Normalización fija:** se usan $\boldsymbol{\mu}$ y $\boldsymbol{\sigma}$ de CIFAR-10 incluso al entrenar con imágenes propias. Funciona, pero no es óptimo para conjuntos de imágenes muy diferentes.
- **Inferencia síncrona:** la petición espera a que termine la clasificación, y el endpoint de lote procesa las imágenes una por una.
- **Sin reanudación del entrenamiento:** el checkpoint no guarda el estado del optimizador ($\mathbf{b}_t$) ni la época, así que un entrenamiento interrumpido no puede continuar donde se quedó.
- **SQLite:** adecuado para desarrollo y cargas pequeñas, pero no para muchas escrituras concurrentes.

13.2 Mejoras sugeridas

- Añadir autenticación por token y *throttling* en DRF.
- Calcular $\boldsymbol{\mu}$ y $\boldsymbol{\sigma}$ del conjunto propio durante el entrenamiento y guardarlas en el checkpoint.
- Guardar optimizador, planificador y época en el checkpoint para poder reanudar el entrenamiento.
- Procesar los lotes de imágenes en un único tensor para aprovechar mejor la GPU.
- Definir un umbral mínimo de confianza para responder «desconocido» cuando $\max_k p_k$ sea bajo.
- Registrar métricas por época (por ejemplo con TensorBoard) y añadir parada temprana.
- Generar documentación OpenAPI/Swagger de la API.
- Empaquetar la API en un contenedor Docker para facilitar el despliegue.

14. Glosario

Aumento de datos (data augmentation)
Transformaciones aleatorias (recortes, espejos) aplicadas a las imágenes de entrenamiento para que el modelo generalice mejor.

Backpropagation
Algoritmo que calcula el gradiente $\nabla_\theta \mathcal{L}$: cómo contribuye cada peso al error, para ajustarlo en la dirección que lo reduce.

Checkpoint
Archivo con los pesos aprendidos y la configuración del modelo, que permite reutilizarlo sin volver a entrenar.

Confianza
Probabilidad asignada por el modelo a la clase más probable, $\max_k p_k$.

Entropía cruzada
Función de pérdida $-\log p_y$ que penaliza al modelo cuando asigna baja probabilidad a la clase correcta.

Época
Una pasada completa por todas las imágenes de entrenamiento.

Endpoint
Combinación de método HTTP y ruta que ofrece una operación concreta de la API.

Gradiente
Vector de derivadas parciales de la pérdida respecto a cada peso; indica la dirección de mayor aumento del error.

Logits
Puntajes sin normalizar $\mathbf{z}$ que produce la última capa; softmax los convierte en probabilidades.

Lote (batch)
Grupo de N imágenes que se procesan juntas en un paso de entrenamiento.

MPS
Metal Performance Shaders: el backend de PyTorch que usa la GPU de los Mac con Apple Silicon.

Precisión (accuracy)
Porcentaje de imágenes clasificadas correctamente.

ResNet
Red neuronal residual: arquitectura que añade conexiones de atajo entre capas para poder entrenar redes muy profundas.

Serializer
Componente de DRF que valida los datos de entrada y convierte los objetos en JSON.

Softmax
Función $p_k = e^{z_k} / \sum_j e^{z_j}$ que transforma un vector de puntajes en probabilidades positivas que suman 1.

Tasa de aprendizaje (learning rate)
Tamaño del paso η con el que se actualizan los pesos en cada iteración.

Top-k
Las k clases con mayor probabilidad, ordenadas de mayor a menor.

Validación
Conjunto de imágenes que no se usan para entrenar y sirven para medir cómo generaliza el modelo.

Weight decay
Regularización $\lambda\theta$ que penaliza los pesos grandes para reducir el sobreajuste.